

Course Code: CSE440

Section: 02

Group No: 7

Group Members:

Name: Kazi Refaet Ullah, ID: 2131510642

Name: Sharia Sakib, ID: 2021270642

Name: Samiul Ahamed Fhyas, ID: 2013221642

Name: Md Asaduzzaman Limon, ID: 1931374042

1. Overview

This project is a Texas Hold'em poker application built around two complementary pieces of work: a rigorous AI decision-making algorithm developed and validated on a simplified poker variant, and a fully playable, polished four-player Texas Hold'em web game built on top of it. The two pieces are connected by a shared theme — decision-making under uncertainty and against opponents — but were deliberately scoped differently to fit what each was good for.

The research component uses Leduc Hold'em, a small two-player poker variant with only three card ranks, as a controlled environment for implementing and benchmarking Expectiminimax search with alpha-beta pruning. The playable component is a four-seat Texas Hold'em table where a human player competes against three distinct AI personalities, built with Streamlit and given a custom, casino-table-styled interface.

1.1 Project Objectives and Scope

The main objective of the project is to demonstrate how artificial intelligence techniques can be applied to an imperfect-information game, while also producing a playable software artifact that is understandable and enjoyable for users. Poker is an appropriate domain because every decision must be made with incomplete knowledge: a player knows their own cards and the public board, but must infer opponent strength from betting behavior, position, pot size, and risk. This makes the project more challenging than a deterministic board game such as tic-tac-toe or chess, because the agent must reason not only about the current visible state but also about many possible hidden states.

The scope was intentionally divided into a research-oriented component and a product-oriented component. The research side focuses on a small poker environment where a formal search algorithm can be tested carefully. The product side focuses on a richer, four-player game where usability, visual clarity, and real-time interaction become more important. This separation allows the project to be both academically meaningful and practically demonstrable, rather than overextending a single algorithm into a setting where it would become too slow or theoretically inappropriate.

1.2 Key Design Decisions

Several design decisions shaped the final implementation. First, the project uses Leduc Hold'em as the formal evaluation environment because its smaller deck and simpler betting rules make exhaustive or depth-limited analysis feasible. Second, the web application uses rule-based AI for the full Texas Hold'em table because a multiplayer expectiminimax tree would grow too quickly and would not provide the responsive experience expected from an interactive app. Third, the interface is treated as part of the project outcome rather than as a secondary detail; the visual table, seat placement, status indicators, and action history are all designed to make the system easier to understand while it is running.

The topic also fits the course context because it combines algorithm design, probability, state representation, and software implementation. A poker AI cannot simply follow one fixed path; it must evaluate uncertain outcomes and make decisions that remain reasonable even when the exact opponent hand is unknown. This makes the project a practical example of how artificial intelligence techniques move from theory into an application where speed, usability, and correctness must all be considered together.

2. Project Files

The repository contains four Python files:

File	Purpose
leduc_env.py	Defines the Leduc Hold'em environment: a simplified two-player, 3-rank poker game used as a controlled test bed for game-tree search (state representation, legal actions, betting rules, and payouts).
agents_and_sim.py	The algorithmic core. Implements Expectiminimax search with alpha-beta pruning for the Leduc game, plus baseline agents (Random, Call Station, Tight-Aggressive) and a simulation harness that benchmarks Expectiminimax against each baseline over hundreds of hands, including a search-depth ablation study.
play_vs_ai.py	A terminal interface to play Leduc Hold'em directly against the Expectiminimax agent.
streamlit_app.py	The main deliverable: a polished four-seat Texas Hold'em web app (built with Streamlit) where a human player faces three rule-based AI opponents. This file contains all of the UI/UX design work.

2.1 Repository Organization and Maintainability

The repository is organized so that the algorithmic work and the interactive game can be understood separately. This is important because the Leduc environment and the Streamlit game solve different problems. The Leduc files form a compact experimental framework: the environment defines the rules, the agents choose actions, and the simulation code measures performance. The Streamlit file contains a larger amount of application logic because it must manage game state, interface rendering, user input, animations, and round transitions within Streamlit's rerun-based execution model.

From a maintainability perspective, the cleanest future improvement would be to separate the Streamlit game into smaller modules: one module for card and hand evaluation, one for betting and pot distribution, one for AI decisions, and one for visual rendering. The current single-file approach is convenient for a course project because it makes deployment simple, but a modular structure would make debugging, testing, and future feature development easier.

3. AI Algorithm and Research Component

The academic core of the project is an Expectiminimax search agent, implemented in `agents_and_sim.py` against the Leduc Hold'em environment in `leduc_env.py`.

3.1 Why Leduc Hold'em

Leduc Hold'em is a standard simplified poker benchmark in AI game theory research: two players, a 6-card deck (three ranks, two suits each), one shared board card, and two betting rounds with a fixed raise size and a cap of two raises per round. It is small enough to search exhaustively while still containing the two properties that make poker hard for a game-tree algorithm: hidden information (each player only sees their own hole card) and chance events (the board card draw).

Although Leduc Hold'em is much smaller than Texas Hold'em, it still preserves the central strategic difficulty of poker. A player may have a strong private card, but the public board can change the relative strength of hands, and the opponent may act in a way that suggests confidence or weakness. Because of this, the environment is useful for studying how an AI agent should reason when the available information is incomplete. It allows the algorithm to be tested in a controlled setting before applying related ideas to a larger game application.

3.2 Expectiminimax with alpha-beta pruning

Expectiminimax generalizes minimax to games with chance nodes. The search tree has three kinds of nodes:

- Max nodes — the agent's own decision points, where it picks the action that maximizes its expected value.
- Min nodes — the opponent's decision points, treated adversarially (the opponent is assumed to minimize the agent's value).
- Chance nodes — the board card reveal, where the value is the probability-weighted average over every card that could be dealt.

Alpha-beta pruning is layered onto the max and min nodes to cut off branches that cannot affect the final decision, reducing the number of nodes expanded without changing the result. Because the agent cannot see the opponent's hole card, `ExpectiminimaxAgent.act()` first enumerates every card the opponent could plausibly be holding (weighted by how many of that rank remain in the deck), runs a full expectiminimax search for each hypothesis, and averages the results — effectively marginalizing over the hidden information before committing to an action.

A depth-limited heuristic (`evaluate_heuristic`) estimates a state's value by calculating the agent's equity (win probability) against the remaining possible opponent cards and combining it with the current pot size, allowing the search to be cut off early when full-depth search is too expensive.

The heuristic evaluation is especially important because poker games can branch quickly even in a simplified environment. At each point, the agent may need to consider folding, calling, checking, or raising; after that, the opponent can respond, and chance events may still occur. Without a heuristic cutoff, the search can become slow as depth increases. The evaluation function therefore acts as a practical compromise: it gives the agent an estimate of how favorable the current situation is without requiring every possible future action sequence to be explored completely.

3.3 Baselines and evaluation

Three baseline agents are implemented for comparison: a `RandomAgent` that acts uniformly at random, a `CallStationAgent` that always calls or checks, and a `TightAggressiveAgent` that only plays strong hands aggressively and folds otherwise. The `simulate()` function runs the Expectiminimax agent against each baseline for hundreds of hands (alternating which seat each agent occupies, to cancel out positional advantage), and reports average net chips per hand, total wall-clock time, and nodes expanded. A separate depth-ablation experiment reruns the matchup at search depths of 2, 4, 8, and 20 to show the trade-off between search depth, decision quality, and computation time.

The baseline agents are useful because they create clear comparison points. The `RandomAgent` shows whether the search agent can beat completely unstructured behavior. The `CallStationAgent` tests whether the search agent can profit against a player who rarely applies pressure but continues too often. The `TightAggressiveAgent` is a stronger comparison because it folds weaker hands and raises stronger ones. If Expectiminimax performs well against all three, it suggests that the search process is making meaningful decisions rather than only exploiting one weak playing style.

3.4 Why not Expectiminimax for the 4-player game

Expectiminimax is designed for two-player, zero-sum games, where one player's gain is exactly the other's loss. A four-player table with side pots and simultaneous folding decisions is neither two-player nor strictly zero-sum, so the algorithm does not extend to it cleanly. Rather than force a mismatched algorithm onto the interactive game, the project keeps Expectiminimax as the rigorous, benchmarked algorithm on Leduc Hold'em, and uses fast, practical rule-based “personality” agents (described below) for the four-player table, where responsiveness during live play matters more than game-theoretic optimality.

3.6 Performance Considerations

Search depth is one of the main trade-offs in the algorithm. A deeper search can make stronger decisions because it sees farther into future betting rounds and board outcomes, but it also expands more nodes and takes longer to run. The depth-ablation experiment is therefore useful because it shows whether additional search depth produces meaningful gains or whether the cost grows faster than the benefit. This kind of evaluation is important in AI projects because an algorithm that is theoretically stronger may still be impractical if it cannot respond quickly enough for interactive use.

3.5 Handling Uncertainty in the Search Model

The most important technical challenge in the search component is that the agent cannot directly observe the opponent's private card. To address this, the agent does not evaluate only one game state. Instead, it considers a set of possible hidden-card worlds that are consistent with the cards already visible. Each possible world is searched separately, and the final value of an action is calculated as an average across those worlds. This approach is not a perfect substitute for a full belief-state poker solver, but it captures the core idea that good decisions should be robust across multiple possible opponent holdings.

The use of chance nodes also makes the model more realistic than ordinary minimax. When the public board card has not yet been dealt, the algorithm cannot assume one fixed future. It must average over all remaining board cards and evaluate how each possible reveal changes the strength of both players' hands. This is why Expectiminimax is a better fit for poker than plain minimax: the game includes both strategic choices and random events, and the agent must reason across both.

4. The Four-Player Game Engine

4.1 Core Game Flow

`streamlit_app.py` implements a full ante-based, fixed-limit Texas Hold'em table for four players (one human, three AI). Each player starts with 100 chips. Every hand, all players ante 1 chip; betting is 2 chips per raise pre-flop and on the flop, and 4 chips per raise on the turn and river, with a maximum of two raises per betting round. The engine deals two private hole cards to each player and reveals the shared board in the standard flop (3 cards), turn (1 card), and river (1 card) sequence.

The game loop follows the standard sequence of a poker hand. A new hand begins by resetting temporary state, collecting antes, shuffling the deck, dealing hole cards, and setting the first active player. The game then advances through betting rounds, revealing community cards only after all active players have either matched the current bet, folded, or gone all-in. This flow prevents premature card reveals and keeps the player's decisions aligned with the rules of fixed-limit Hold'em.

State management is especially important in Streamlit because the app reruns from top to bottom after every user interaction. The project uses persistent session state to remember chip stacks, player statuses, hole cards, board cards, pot contributions, current phase, and action history between reruns. Without this structure, every button press would effectively restart the app. By storing game variables consistently, the application can behave like a continuous game even though the underlying framework refreshes the script repeatedly.

A reliable state model also helps the program avoid common poker-game errors. For example, the system must know which players are still active, which players have already folded, how much each player has contributed to the current pot, and whether the next action should advance the betting round or move directly to showdown. These details may seem small, but they are essential for making the game feel fair. If chip counts or player turns are handled incorrectly, the user quickly loses trust in the application.

The engine correctly supports all-in play: a player who cannot cover a full raise may still call all-in but cannot raise for less than the fixed size, remaining board cards are dealt automatically once betting is settled, and the pot is split into a main pot and side pots so that each pot is only contested among the players who were able to match it (`pot_layers` and `distribute_pots`). Hand strength is evaluated with a full five-card hand evaluator (`five_card_rank / best_five_hand`) that checks all $C(7,5)$ combinations of hole and board cards to find each player's best possible hand, supporting every standard poker hand category from high card through straight flush.

4.2 Betting Logic and Pot Integrity

The betting system is designed to preserve fairness even when players have different chip counts. Each player's contribution is tracked so that the program can determine how much they still owe to call and whether they are eligible to win each portion of the pot. This prevents a short-stacked player from winning chips they did not risk, while still allowing them to remain in the hand for the portion they could afford. The side-pot logic therefore supports one of the most important edge cases in multiplayer poker.

Another important detail is raise capping. Fixed-limit poker normally restricts the number of raises per betting round, and this project follows that structure with a two-raise limit. This keeps hand length manageable, avoids endless betting loops, and makes AI behavior easier to predict and test. It also creates clear strategic pressure: once the cap is reached, players must decide whether to call, fold, or continue to showdown without the possibility of further escalation in that round.

The pot logic is one of the most important parts of the implementation because poker outcomes are based not only on who has the best hand, but also on how much each player has risked. Side pots are needed whenever one or more players run out of chips before others finish betting. In that situation, the program must divide the total pot into layers and identify which players are eligible for each layer. This ensures that a player who is all-in for a smaller amount can still win the main pot, while the remaining players continue competing for the extra chips they placed after that point.

4.1 The three AI personalities

Instead of a game-tree search, the three opponents (`ai_action`) use a fast heuristic: a numeric hand-strength estimate (based on card rank, pairs, and matches with the board) is compared against a per-personality raise/fold threshold and a random roll, so each personality reacts differently to the same hand strength:

- Atlas (aggressive) — raises more often and folds rarely, even with a low raise threshold of 0.30 and a low fold threshold of 0.10.
- Nova (strategic) — balanced thresholds (0.52 raise, 0.28 fold) that adapt more evenly to hand strength.
- Echo (careful) — raises only with strong hands (0.72 threshold) and folds weak hands more readily (0.48 threshold).

4.3 AI Personality Rationale

The three personalities are not meant to be perfect poker players. Their purpose is to make the table feel varied and readable. Atlas creates pressure by raising frequently, which forces the human player to decide whether to continue with marginal hands. Nova provides a balanced opponent that can call, raise, or fold depending on estimated strength. Echo adds contrast by being conservative, making its raises feel more meaningful. Together, the personalities create a more interesting experience than three identical bots because the user can begin to form expectations about each seat's style.

Randomness is also intentionally included in the AI decision rule. If each opponent made the same action every time a hand-strength threshold was crossed, the game would become too predictable. By combining thresholds with controlled randomness, the AI can occasionally surprise the player while still remaining consistent with its overall personality. This balance helps the game feel less scripted without requiring a computationally expensive strategic solver.

This personality-based approach also improves the educational value of the application. A user can observe how different strategies behave under the same rules and compare their own decisions against each opponent type. For example, an aggressive opponent may win small pots by forcing folds, while a careful opponent may enter fewer hands but show stronger holdings at showdown. These patterns help demonstrate that poker decision-making is not only about card strength; it is also about betting behavior, risk tolerance, and reading patterns over repeated hands.

5. UI / UX Design

The interactive experience is the project's most design-heavy component. Rather than presenting Streamlit's default form-style layout, the app is styled from scratch to resemble a virtual casino table, using custom HTML and CSS

injected into the page.

5.1 Visual theme

The page background is a radial green-to-black gradient evoking felt under low light, and the table itself is a large rounded rectangle with a wood-colored border and a gold inset stroke, styled with box-shadows to suggest physical depth. Streamlit's own default chrome — the hamburger menu, footer, and header — is hidden so the app reads as a standalone product rather than a Streamlit demo. Playing cards are drawn as custom HTML/CSS elements (rounded rectangles with a rank and suit, red-colored for hearts and diamonds) rather than plain text, and hidden cards are rendered with a diagonal-striped card-back pattern.

The visual design also supports understanding. Since poker involves several pieces of information at once, the interface must prevent the player from feeling lost. Showing the table, community cards, pot, chip stacks, and player statuses in consistent positions reduces the mental effort required to follow the hand. This is important because the user should spend attention on making strategic decisions, not on trying to interpret the interface.

5.2 Table layout

A CSS grid arranges the four seats spatially the way a real table would: two AI seats sit at the top corners, a third AI seat sits centered above the pot, and the human's own seat spans a wide strip across the bottom — mirroring how a player looks across the table at their opponents. The pot total and the community cards sit together in the center of the table, labeled with the current betting phase (pre-flop, flop, turn, or river) so the game state is always visible without scrolling or clicking.

5.3 State feedback and motion

The seat currently acting is highlighted with a pulsing glow animation so the player's eye is drawn to whoever is deciding. Cards animate in with a brief deal motion when they are revealed. When an AI is deciding, an animated three-dot “thinking” indicator appears next to its name, paired with an actual timed delay (configurable from half a second to three seconds via a sidebar slider) so AI turns feel paced rather than instantaneous. Every seat displays a compact status tag — folded, all-in, your turn, thinking, or in hand — so the full state of the table is readable at a glance.

5.4 Feedback and history

A single-line “latest action” banner below the table always shows the most recent event in plain language (for example, a call, a raise, or a phase reveal), and a collapsible “hand activity” expander preserves the complete log for players who want the full play-by-play. At showdown, the app also reveals each live player's hole cards and a plain-language description of their best hand (for example, “Kings full of eights”), and, when a hand produced side pots, lists exactly how each pot was split and to whom.

The action history is particularly helpful for learning and debugging. During a hand, a player may forget whether an opponent checked, called, or raised earlier in the round. By keeping a visible log, the application allows the player to review the sequence of decisions and understand why the pot reached its current size. From a development perspective, the same history can help identify logic errors because every major state transition is recorded in plain language.

5.5 Controls

A sidebar keeps controls and reference information out of the way of the table itself: live chip stack, hands played, cumulative session result, the AI thinking-time slider, buttons to start a new hand or reset the whole table, and short static explanations of the betting rules and opponent personalities. When it is the human's turn, contextual action buttons appear directly beneath the table (fold, check, call, raise); the call button is dynamically labeled with the exact amount owed, or relabeled “all-in” when the player cannot cover a full call, so the cost of each decision is explicit before it is made.

5.6 Responsiveness

A mobile media query collapses the two-column seat grid into a single column, shrinks card sizes, and hides secondary seat labels below a 700px viewport width, so the table remains playable on a phone-sized screen rather than only on desktop.

6. Testing and Validation

Testing for this project can be considered in two categories: algorithmic validation and gameplay validation.

Algorithmic validation checks whether the Leduc environment follows the expected rules, whether legal actions are generated correctly, whether terminal payouts are calculated accurately, and whether Expectiminimax improves over weaker baseline agents. Gameplay validation checks whether the four-player app behaves correctly from a user's perspective: buttons should appear only when valid, chip totals should remain consistent, community cards should reveal in the correct order, and folded or all-in players should no longer be asked to act.

Important edge cases include hands where multiple players go all-in with different stack sizes, hands where everyone folds except one player, showdowns with tied hands, and rounds where the raise cap is reached. These situations are more likely to reveal bugs than ordinary hands because they stress the pot-distribution and turn-advancement logic. A useful validation strategy is to create deterministic test hands with known outcomes and compare the program's result against the expected winner and chip movement.

User testing is also valuable because interface problems are not always visible from code inspection. A player should be able to understand whose turn it is, how much it costs to call, what phase the hand is in, and what happened most recently. If users repeatedly ask the same question while playing, that is a signal that the interface needs a clearer label, better placement, or stronger visual feedback.

Validation should also include repeated-play testing rather than only testing a single hand. Some defects appear only after several hands, especially when session counters, chip stacks, reset buttons, and player statuses carry over from one round to the next. A strong testing process would therefore involve playing many consecutive hands, recording whether the total number of chips remains constant, and checking whether the app correctly returns to a clean state at the beginning of each new hand.

7. Limitations

The main limitation of the project is that the strongest search algorithm is demonstrated in the simplified Leduc environment rather than in the full four-player Texas Hold'em game. This is a reasonable design choice, but it means the web game's opponents are not game-theoretically optimal. Their decisions are based on hand-strength estimates, thresholds, and randomness rather than on a full opponent model or long-term expected-value calculation.

Another limitation is that the four-player game does not appear to include long-term learning. The AI personalities do not adapt based on the human player's previous behavior, bluff frequency, or calling tendencies. This keeps the implementation transparent and fast, but it also limits strategic depth over a long session. A more advanced version could maintain statistics about the human player and adjust opponent thresholds over time.

The project also prioritizes clarity over complete casino realism. Some poker details, such as complex blind structures, variable bet sizing, tournament elimination rules, and advanced opponent ranges, are outside the current scope. However, this simplified rule set is appropriate for an educational project because it keeps the mechanics understandable while still preserving the essential decision-making structure of poker.

A further limitation is that the AI does not explicitly model bluffing as a separate strategic concept. In real poker, a player may raise with a weak hand to represent strength or call with a medium hand to control the pot. In this project, AI behavior is mostly tied to estimated hand strength and personality thresholds. While randomness creates some unpredictability, a dedicated bluffing model would make opponent behavior more realistic and strategically deeper.

8. Future Work

Future work could strengthen both the AI and the application design. On the AI side, the project could add Monte Carlo simulation for the four-player game, allowing each opponent to estimate winning chances by sampling many possible unseen cards rather than relying only on simple thresholds. A more ambitious extension would be counterfactual regret minimization, which is commonly used in poker research because it is better suited to imperfect-information games than direct minimax-style search.

On the software side, future work could include a replay system, saved session statistics, difficulty settings, and a clearer separation between the game engine and the visual layer. The replay system would be especially useful for debugging and demonstration because it would allow a completed hand to be stepped through action by action.

Saved statistics could also make the experience more engaging by showing the player's long-term win rate, biggest pot won, most profitable opponent matchup, and overall chip movement over time.

Another useful extension would be a difficulty system. Beginner mode could make the opponents more transparent by using simple thresholds and fewer aggressive raises. Intermediate mode could use the current personality system. Advanced mode could combine opponent modeling, Monte Carlo sampling, and limited bluffing behavior to create a more challenging table. This would allow the project to serve different users: casual players, students learning poker concepts, and evaluators interested in stronger AI behavior.

9. Conclusion

The project pairs a small, theoretically clean two-player poker variant — used to properly implement and benchmark Expectiminimax search under uncertainty — with a much larger, four-player Texas Hold'em game whose AI opponents trade search-based optimality for speed and personality, and whose interface was deliberately designed to feel like a real, responsive card table rather than a bare data-entry form. The two halves reflect a good-faith scoping decision: pursue algorithmic rigor where the game structure supports it, and pursue a strong, playable user experience where it does not.

Overall, the project demonstrates a strong connection between artificial intelligence concepts and software engineering practice. The Leduc component shows that the student understands game-tree reasoning, uncertainty, chance events, heuristic evaluation, and benchmark comparison. The Texas Hold'em application shows that the same domain knowledge can be translated into a usable product with an attractive interface, readable feedback, and complete multiplayer hand flow. Although there is room for future improvement, especially in adaptive AI and modular software design, the current version successfully presents both the technical foundation and the interactive experience of a poker AI project.